计算机操作系统

Operating Systems

田卫东

March, 2014

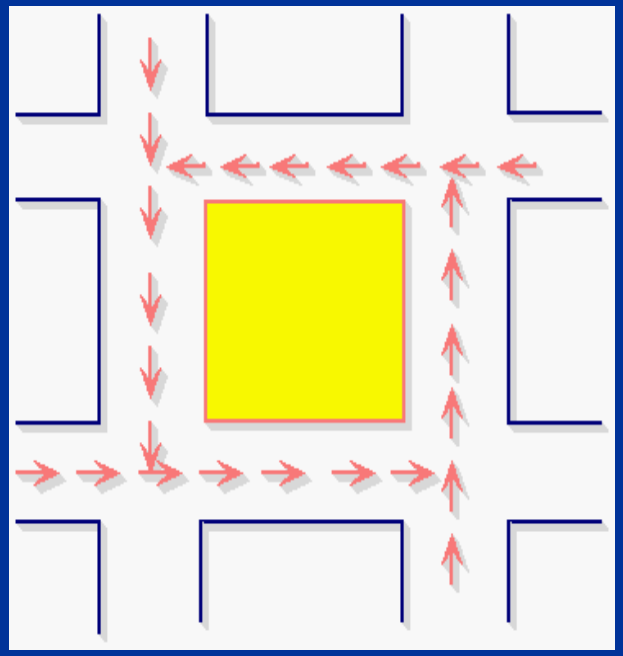
第3章 处理机调度与死锁

---进程死锁专题

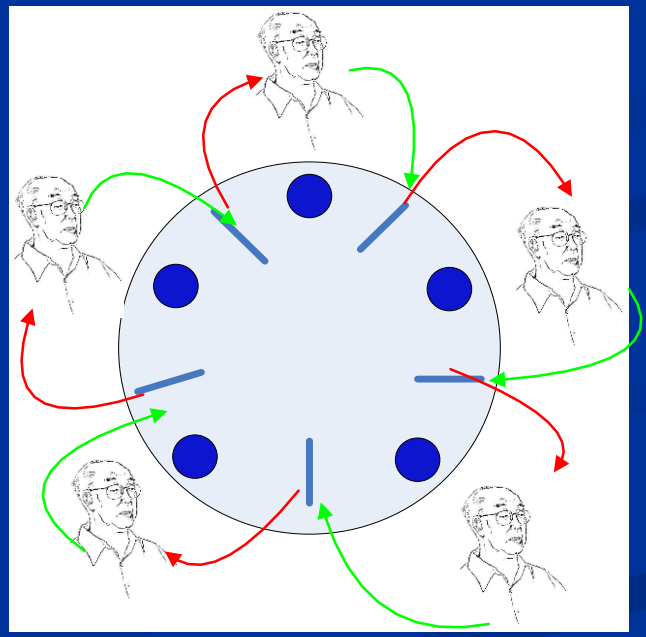
3.5 死锁的基本概念

3.5.1 死锁及死锁状态

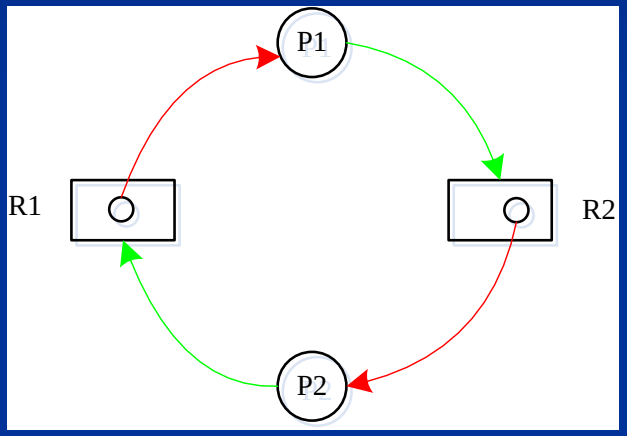
例1：交通死锁



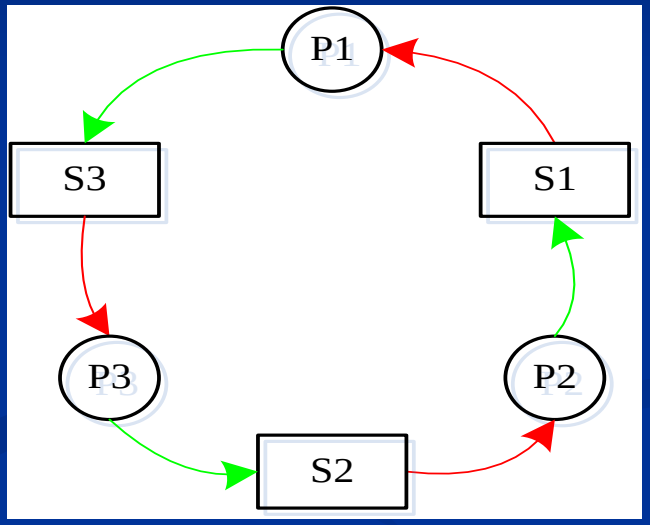
例2：哲学家进餐



例3：两进程共享资源



例4：三进程接发消息



P1：Receive(S3), Send(S1)

P2：Receive(S1), Send(S2)

P3：Receive(S2), Send(S3)

- 死锁（Deadlock）的定义：

  指多个进程在运行过程中因争夺资源而造成的一种僵局。当进程处于这种僵局时，若无外力作用，他们都将无法再向前推进。

- 死锁与阻塞的区别

  死锁的进程处于阻塞状态，但仅依靠自己，无法继续运行。

从并发进程的代码看死锁1

生产者-消费者问题：

```
VAR mutex, empty, full: semophore := 1, n, 0 ;
    In,out: integer := 0, 0 ;
    Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
   begin
       Repeat
          Produce an item in nextp ;
          ……
          Wait( empty ) ;
          Wait( mutex ) ;
          Buffer(in) := nextp ;
          in := (in + 1 ) mod n ;
          Signal( mutex ) ;
          Signal( full ) ;
       Until flase
   End
```

```
Consumer:
   begin
       Repeat
          Wait( full ) ;
          Wait( mutex ) ;
          Nextc = Buffer(out);
          out := (out + 1 ) mod n ;
          Signal( mutex ) ;
          Signal( empty ) ;
          Consume the item nextc ;
       Until flase
  End
Parend
```

从并发进程的代码看死锁1

生产者-消费者问题(错误代码)：

```
VAR mutex, empty, full: semophore := 1, n, 0 ;
        In,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
    begin
          Repeat
             Produce an item in nextp ;
             ……
             Wait( empty ) ;
             Wait( mutex ) ;
             Buffer(in) := nextp ;
             in := (in + 1 ) mod n ;
             Signal( mutex ) ;
             Signal( full ) ;
          Until flase
    End
```

```
Consumer:
    begin
          Repeat
             Wait(mutex) ;
             Wait(full) ;
             Nextc = Buffer(out);
             out := (out + 1 ) mod n ;
             Signal( mutex ) ;
             Signal( empty ) ;
             Consume the item nextc ;
          Until flase
    End
Parend
```

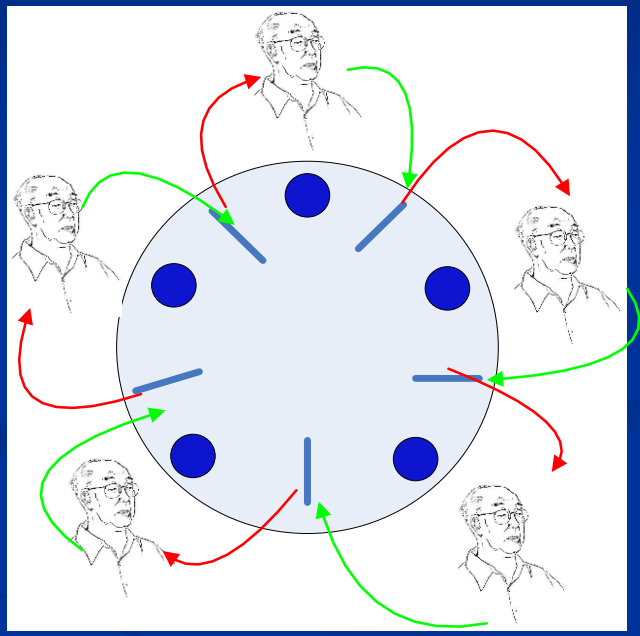
出现死锁时机：

empy=n and full = 0 ;

从并发进程的代码看死锁2

哲学家进餐问题(错误代码):

```
VAR chopstick: array [0..4] of semophore := 1 ;
Parbegin
  Philospheri:
    begin
          Repeat
             Think ;
             Wait(chopstick[i] ) ;
             Wait(chopstick[(i+1)mod 5] ) ;
             …
             Eat ;
             Signal(chopstick[i] ) ;
             Signal(chopstick[(i+1)mod 5] ) ;
        Until flase
    End
Parend
```



出现死锁时机:

执行第一个Wait后;

3.5.2 产生死锁的原因

- 竞争非剥夺性资源

  可剥夺性资源（未用完可以被剥夺）：内存等。

  非剥夺性资源（必须在用完后才能被剥夺）：打印机等

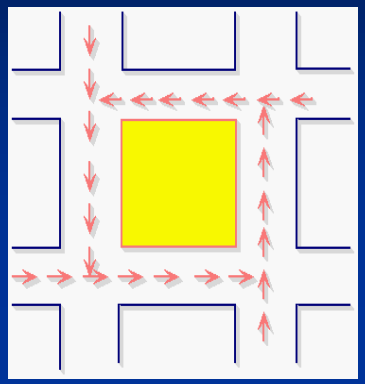
- 进程推进顺序非法

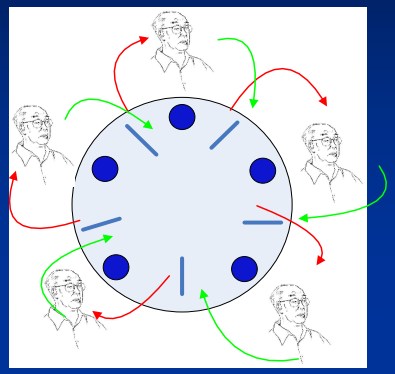
  错误的进程推进顺序 导致 死锁。

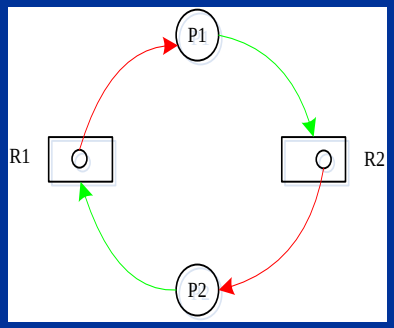
  正确的进程推进顺序 不导致 死锁。

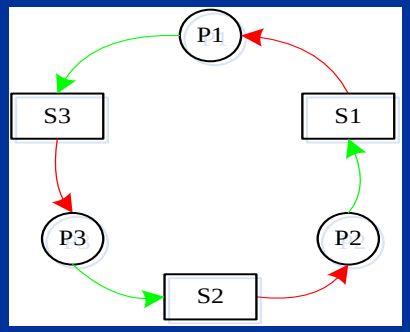
  示例：黑板。

3.5.3 产生死锁的必要条件-死锁的特征









- 互斥条件

  互斥使用资源：资源自身的特点

- 请求和保持条件

  进程占有一个资源的同时，请求另外的资源。

- 不剥夺条件

  进程的资源使用完之前，不能被剥夺。

- 环路等待条件

  死锁的n个进程形成环形的进程-资源链。

  <P1→P2 → … → Pi → … → Pn>

3.5.4 处理死锁的基本方法

- 预防死锁

  作法：破坏死锁产生的四个必要条件

  优点：简单；

  缺点：资源利用率低，效率低

- 避免死锁

  作法：进程申请资源时，由系统审查申请的合理性，只有不导致死锁的申请，才被认可。

  优点：效率略高。

  缺点：实现困难：进程对资源的需求不好事先确定；

- 检测和解除死锁

  作法：进程申请资源时不作任何限制，仅在系统中定时或资源不足时，检查是否有死锁；如果有，解决之。

  优点：不影响系统执行性能，效率高。

3.6 预防死锁

3.6.1 预防死锁的基本概念

- 基本思路

  死锁 → 四个必要条件

  逆否命题成立：

  四个必要条件不同时成立 → 无死锁

- 作法

  破坏死锁产生的四个必要条件。

3.6.2 破坏“互斥条件”

- 作法

  不可能：资源的特性决定，否则程序运行结果不可再现

3.6.3 破坏“请求和保持条件”

- 作法

  进程创建时，一次申请所有资源。成功则运行，否则阻塞。

- 优点

  简单、易于实现，安全。

- 缺点

  资源严重浪费；

  进程执行进度大大延迟。

3.6.4 破坏“不剥夺条件”

- 作法

  进程申请资源时，成功则运行，否则释放所有资源后阻塞。

- 优点

  无。

- 缺点

  资源严重浪费；

  代价太大；

  进程执行进度严重延迟。

3.6.5 破坏“环路等待条件”

- 作法

为所有资源编号，进程申请资源时必须按序申请资源。

例如：

R1： 打印机, R2: 磁带机； R3: 磁盘； R4:输入机

进程P1使用资源顺序:输入机,磁带机,打印机

但是P1 资源申请 顺序必须为：

R1,R2,R4.

- 优点

资源利用率、系统吞吐量显著提高。

- 缺点

资源还是会浪费；资源编号困难；新资源加入困难。

3.7 避免死锁

避免死锁的基本思想：

- 允许系统具备四个必要条件
- 进程在执行过程中动态地申请和释放资源
- 每次资源分配时，由系统仔细检查此次资源分配的安全性，只有在安全时才分配，否则不分配，并令进程等待。

问题：

如何检测资源分配是否导致死锁？

3.7.1 安全状态与不安全状态

(1) 安全状态和不安全状态

假定某系统有n个进程并发执行，对于某个时刻T0，划分系统安全和不安全的标准如下：

系统能够按照某种进程顺序<P1,P2,…,Pn>执行，当轮到每个进程Pi执行时，都能满足该进程对资源的最大需求，则该系统处于安全状态，否则为不安全状态。

<P1,P2,…,Pn>称为安全序列。

3.7.1 安全状态与不安全状态

(2) 安全状态与不安全状态示例

某系统3个进程P1,P2,P3，共享12台磁带机资源，某时刻T0的资源分配状况如下表所示：

进程	最大需求量	已分配	可用
P1	10	5	3
P2	4	2	
P3	9	2	

1) T0时刻安全性

安全序列：<P2,P1,P3> ➔ 安全状态

3.7.1 安全状态与不安全状态

2) 安全状态转化为不安全状态

进程	最大需求量	已分配	可用
P1	10	5	3
P2	4	2	
P3	9	2	

P3请求1台磁带机

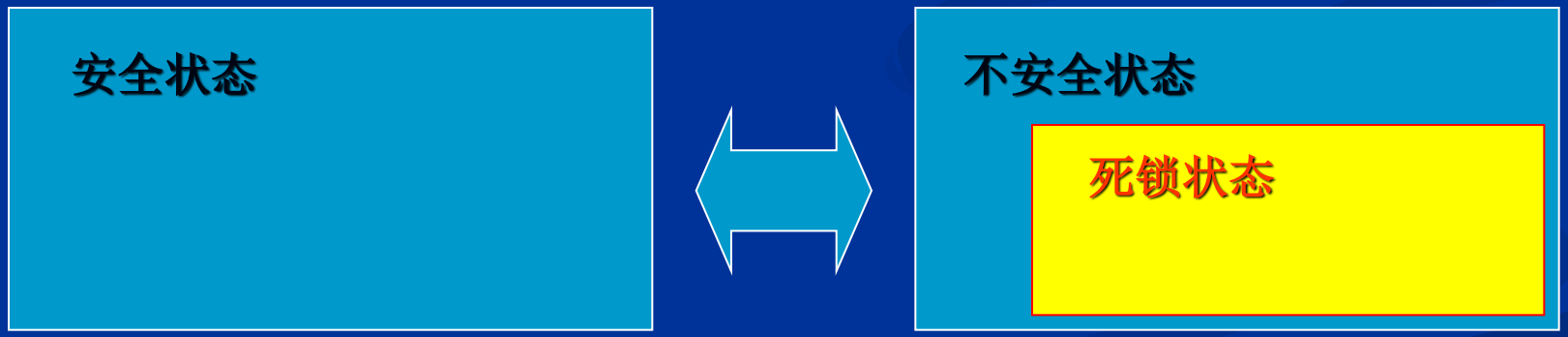
进程	最大需求量	已分配	可用
P1	10	5	2
P2	4	2	
P3	9	3	

安全序列：<P2,？,？> ➔ 不安全状态

3.7.1 安全状态与不安全状态

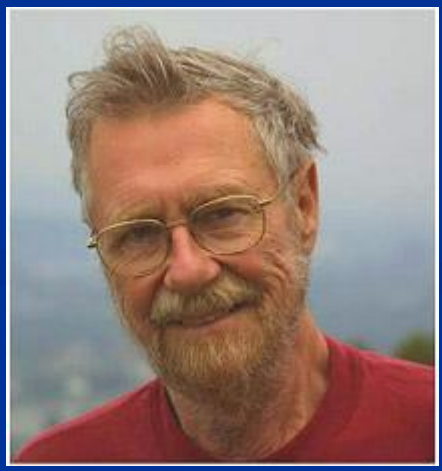
3) 安全状态与不安全状态关系

- 系统处于安全状态，不会死锁；
- 系统处于不安全状态，可能死锁；
- 系统在安全状态与不安全状态之间不断转换



3.7.2 银行家算法

- Dijkstra
  荷兰计算机科学家Edsger Wybe Dijkstra;
  1972年ACM 图灵奖获得者
- 贡献:
  提出“goto有害论”;
  提出信号量和PV原语;
  解决了有趣的“哲学家聚餐”问题;
  最短路径算法(SPF)的创造者;
  第一个Algol 60编译器的设计者和实现者;
  THE操作系统的设计者和开发者;



3.7.2 银行家算法

1. 问题

n个进程{P1,P2,..,Pn}共享m类资源{R1,R2,..,Rm}，进程并发执行时，如何避免死锁。

2. 数据结构设置

- 可用资源向量, Available, 向量长度m

  Available[j]为资源Rj的可用数量；

- 最大需求矩阵， Max, n行m列

  Max[i][j]表示进程Pi对资源Rj的最大需求量;

- 分配矩阵， Allocation, n行m列

  Allocation[i][j]表示进程Pi已分到资源Rj数量;

- 需求矩阵， Need, n行m列

  Need[i][j]表示进程Pi对资源Rj的需求数量;

  Max = Allocation + Need

- 资源请求向量， $Request_i$, 向量长度m

  $Request_i$[j]表示进程Pi对资源Rj的请求数量。

3. 银行家算法（Pi进程提出资源请求$Request_i$ ）

(1) IF $Request_i$ not ≤ $Need_i$ THEN 出错；

(2) IF $Request_i$ not ≤ Available THEN Pi等待；

(3) 试分配，修改数据结构

Allocation,Need,Available；

Available := Available - $Request_i$

$Allocation_i$:= $Allocation_i$ + $Request_i$

$Need_i$:= $Need_i$ - $Request_i$

(4) 执行安全性算法，检查此次资源分配后系统是否处于安全状态；

(5) IF 安全 THEN 正式分配

ELSE 取消试分配， Pi等待；

4. 安全性算法

(1) 设置工作向量Work,长度m,Work := Available ;

(2) 设置状态向量Finish,长度n, Finish := false ;

(3) 从进程集合查找满足下列条件之进程P_k:

Finish[k] = false ;

$Need_k \leq Work$;

IF 未找到这样的进程 THEN GOTO (5)

(4) 执行如下操作:

Work := Work + $Allocation_k$

Finish[k] := true ;

GOTO (3)

(5) IF Finish = true THEN 安全

ELSE 不安全;

合肥工业大学课程《系统工程》2011

3.7.2 银行家算法

4. 银行家算法算例

	Max A B C	**Allocation** A B C	**Need** A B C	**Available** A B C
P0	7 5 3	0 1 0	7 4 3	3 3 2
P1	3 2 2	2 0 0	1 2 2	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

- 问题：

(1) T0时刻安全性；

(2) P1请求$Request_1(1,0,2)$;

(3) P4请求$Request_4(3,3,0)$;

(4) P0请求$request_0(0,2,0)$;

$Request_1(1,0,2)$：预分配结果

	Max	Allocation	Need	Available
	A B C	A B C	A B C	A B C
P0	7 5 3	0 1 0	7 4 3	2 3 0
P1	3 2 2	3 0 2	0 2 0	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

$Request_0(0,2,0)$：预分配结果

	Max A B C	Allocation A B C	Need A B C	Available A B C
P0	7 5 3	0 3 0	7 2 3	2 1 0
P1	3 2 2	3 0 2	0 2 0	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

3.7.2 银行家算法

5. 银行家算法小结

- 提供比死锁预防更好的进程并发度；
- 需要提前预知进程对各类资源的最大需求数量，在很多情况下几乎不可能。

3.7.2 银行家算法

6. 思考题

3个进程共享资源A，每个进程最大需求为2个，则A至少有多少个资源时，才永远不会产生死锁？

n个进程共享资源A，每个进程最大需求为x个，则A至少有多少个资源时，才永远不会产生死锁？

3.8 死锁的检测和解除

检测和解除死锁的基本思想：

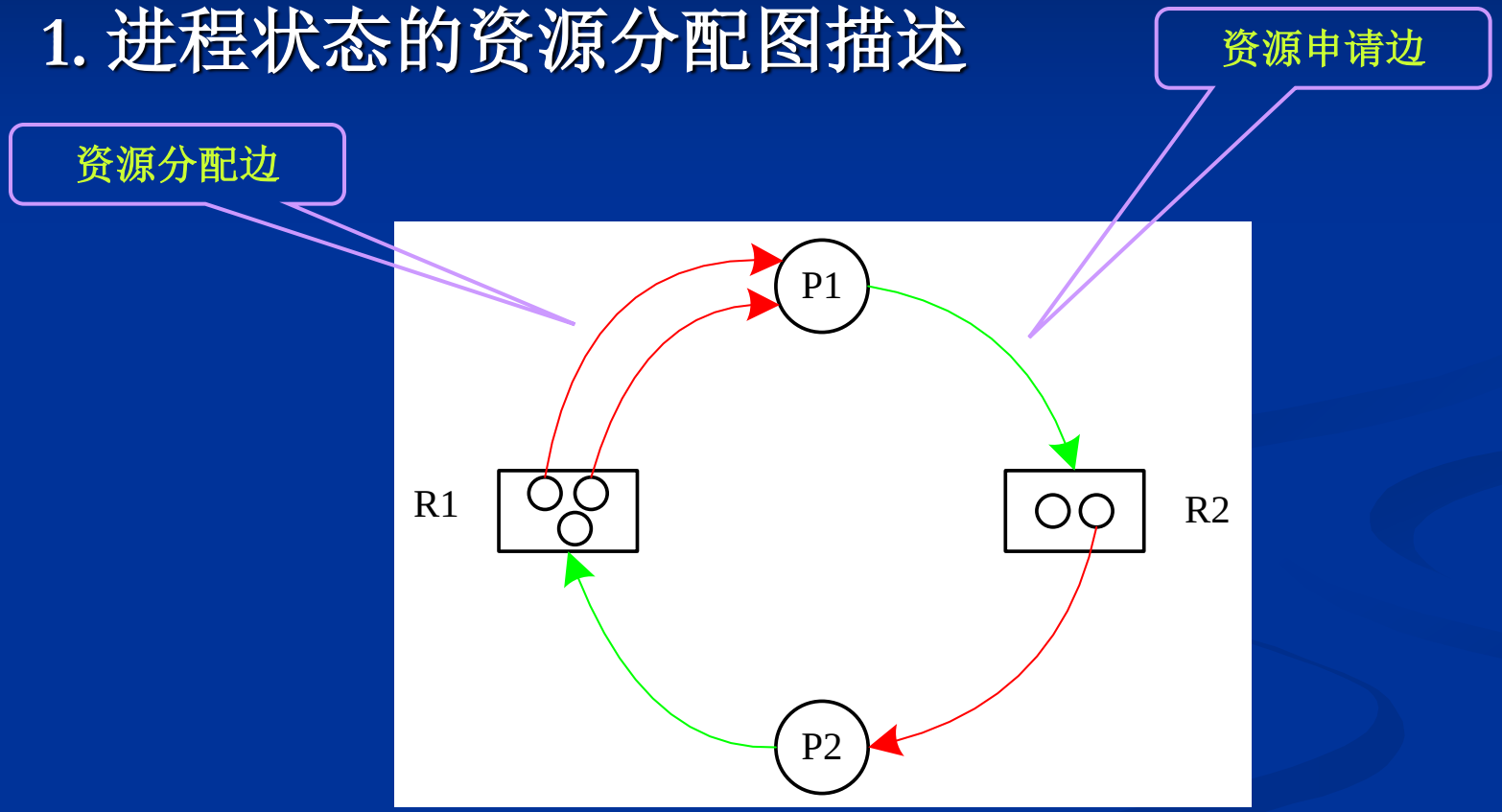
- 进程在执行过程中动态地申请和释放资源
- 每次资源分配时，不作任何检查；
- 允许系统产生死锁；
- 定时或按需检查系统是否有处于死锁状态的进程存在；
- 发现死锁，则立即解除所有死锁进程。

问题：

如何检测处于死锁状态的进程？

3.8.1 死锁的检测

1. 进程状态的资源分配图描述



3.8.1 死锁的检测

2. 死锁检测方法

(1) 对T0时刻资源分配图，寻找一个非阻塞非孤立结点，如找不到，转(4)；

(2) 删除与该结点相连的所有边；

(3) GOTO (1)

(4) IF 最终的资源分配图中都是孤立结点

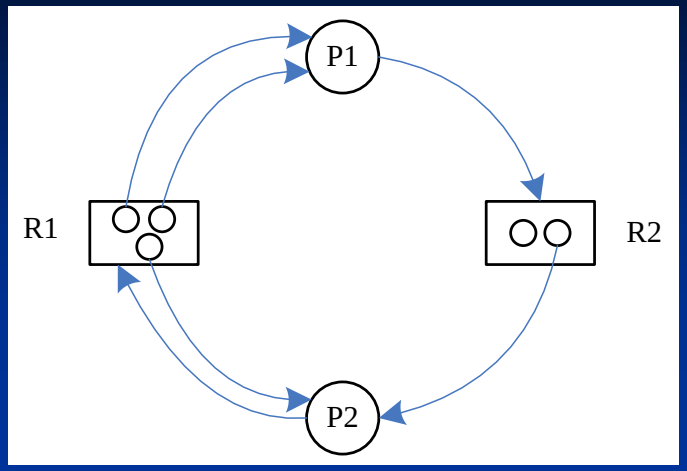
THEN T0时刻资源分配图无死锁

ELSE 有死锁（非孤立结点部分）

死锁定理：

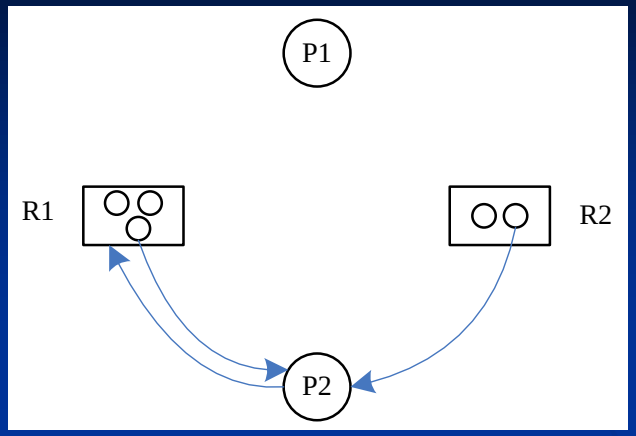
S状态为死锁状态的充分条件是：当且仅当S状态的资源分配图是不可完全简化的。

例1：

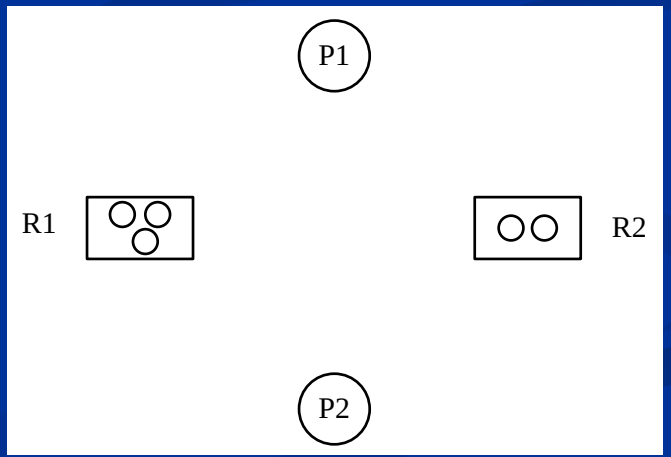


S状态（初始状态）

P1

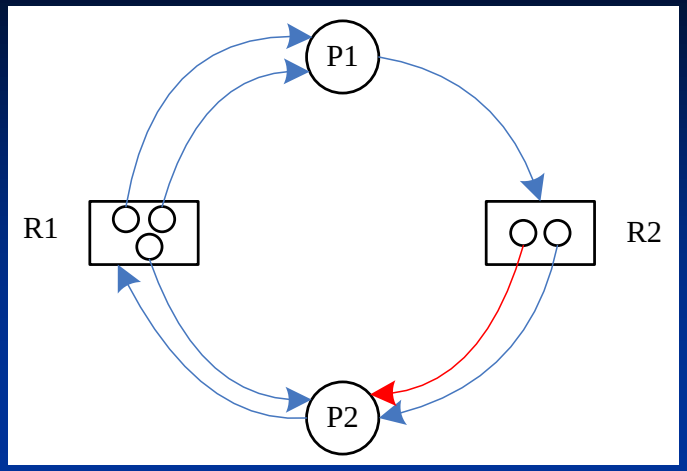


P2



S状态：无死锁

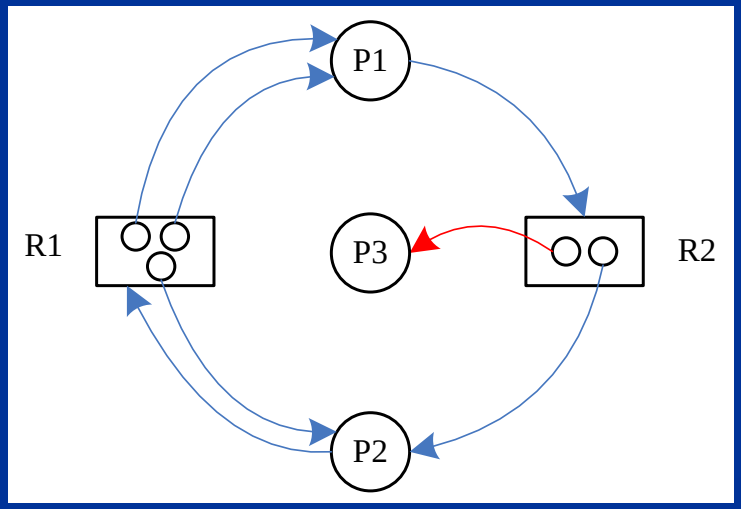
例2：



? → S状态：有死锁

S状态（初始状态）

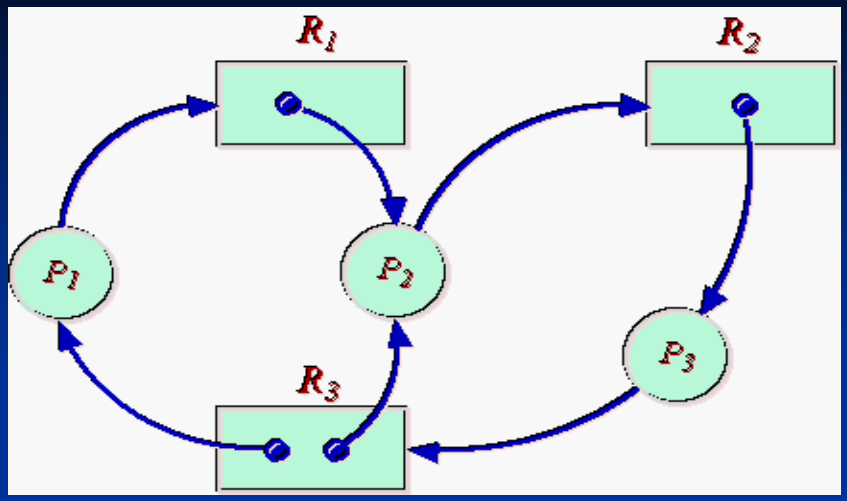
例3：



? → S状态：无死锁

S状态（初始状态）

例4：



?

3.8.1 死锁的检测

3. 思考

如何写成算法？

3.8.2 死锁的解除

- 基本解除策略：

  (1) 剥夺资源；

  (2) 撤消进程；

  - 采用广度进程策略撤消
  - 采用宽度进程策略撤消

本章习题

- P68(修订版教材):
  2,4,6,8,11,17,18,23,25,26,28,33,36,41